부록 · LangChain4j · Spring AI

Spring Boot Java + Kotlin 다음 단계

업비트 실제 XRP 시세 REST API 연동

1. 이번 단계의 목표

이전 단계에서는 XRP 가격을 코드에 직접 입력했습니다. 이번에는 업비트 공개 REST API에서 KRW-XRP 실제 현재가를 가져와 Spring Boot REST API로 출력합니다.

```text
업비트 API
   ↓
실제 XRP 현재가
   ↓
Spring Boot
   ↓
UpbitService
   ↓
XrpController
   ↓
http://localhost:8080/api/xrp
```

이번 단계에서는 JPA, MariaDB, Repository, LangChain4j, Gemma3를 넣지 않습니다. DTO → Service → Controller 흐름부터 확실히 익힙니다.

2. 프로젝트 구조

```text
upbit-ai/
├─ controller/
│  └─ XrpController
├─ service/
│  └─ UpbitService
├─ dto/
│  └─ UpbitTickerDto
└─ UpbitAiApplication
```

핵심 구조: Controller → Service → 외부 Upbit API

3. Java Spring Boot 버전

3-1. UpbitTickerDto.java

```java
package com.wiz.upbit.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public record UpbitTickerDto(
        String market,

        @JsonProperty("trade_price")
        Double tradePrice
) {
}
```

DTO는 업비트가 보내는 JSON 데이터를 Java 객체로 받아주는 '그릇'입니다. trade_price를 Java의 tradePrice에 연결하기 위해 @JsonProperty를 사용합니다.

3-2. UpbitService.java

```java
package com.wiz.upbit.service;

import com.wiz.upbit.dto.UpbitTickerDto;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

@Service
public class UpbitService {

    private final RestClient restClient;

    public UpbitService(RestClient.Builder builder) {
        this.restClient = builder
                .baseUrl("https://api.upbit.com")
                .build();
    }

    public UpbitTickerDto getXrpPrice() {

        UpbitTickerDto[] result = restClient.get()
                .uri("/v1/ticker?markets=KRW-XRP")
                .retrieve()
                .body(UpbitTickerDto[].class);

        return result[0];
    }
}
```

restClient.get()은 업비트 서버에 GET 요청을 보내는 부분입니다. /v1/ticker?markets=KRW-XRP는 원화 기준 XRP 현재가 정보를 요청합니다.

3-3. XrpController.java

```java
package com.wiz.upbit.controller;

import com.wiz.upbit.dto.UpbitTickerDto;
import com.wiz.upbit.service.UpbitService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final UpbitService upbitService;

    public XrpController(UpbitService upbitService) {
        this.upbitService = upbitService;
    }

    @GetMapping
    public UpbitTickerDto getPrice() {
        return upbitService.getXrpPrice();
    }
}
```

4. Kotlin Spring Boot 버전

4-1. UpbitTickerDto.kt

```kotlin
package com.wiz.upbit.dto

import com.fasterxml.jackson.annotation.JsonProperty

data class UpbitTickerDto(
    val market: String,

    @JsonProperty("trade_price")
    val tradePrice: Double
)
```

4-2. UpbitService.kt

```kotlin
package com.wiz.upbit.service

import com.wiz.upbit.dto.UpbitTickerDto
import org.springframework.stereotype.Service
import org.springframework.web.client.RestClient

@Service
class UpbitService(
    builder: RestClient.Builder
) {

    private val restClient = builder
        .baseUrl("https://api.upbit.com")
        .build()

    fun getXrpPrice(): UpbitTickerDto {

        val result = restClient
            .get()
            .uri("/v1/ticker?markets=KRW-XRP")
            .retrieve()
            .body(Array<UpbitTickerDto>::class.java)

        return result!![0]
    }
}
```

4-3. XrpController.kt

```kotlin
package com.wiz.upbit.controller

import com.wiz.upbit.dto.UpbitTickerDto
import com.wiz.upbit.service.UpbitService
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RestController

@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val upbitService: UpbitService
) {

    @GetMapping
    fun getPrice(): UpbitTickerDto {
        return upbitService.getXrpPrice()
    }
}
```

5. Java ↔ Kotlin 비교표

항목	Java	Kotlin
DTO	record	data class

항목	Java	Kotlin
변수	Double tradePrice	val tradePrice: Double
함수	public UpbitTickerDto getXrpPrice()	fun getXrpPrice(): UpbitTickerDto
생성자 주입	생성자를 별도로 작성	클래스 선언부에서 바로 작성
GET 호출	restClient.get()	restClient.get()
@Service	동일	동일
@RestController	동일	동일
특징	명시적이고 코드가 조금 김	간결하고 코드량이 적음

6. 실행 및 확인

Spring Boot를 실행한 뒤 브라우저 또는 Postman에서 다음 주소를 호출합니다.

```text
http://localhost:8080/api/xrp
```

```text
예상 JSON 형태:
{
  "market": "KRW-XRP",
  "tradePrice": 4200.0
}
```

실제 tradePrice 값은 호출 시점의 업비트 XRP 시세에 따라 달라집니다.

7. 꼭 이해해야 할 흐름

```text
브라우저
   ↓
XrpController
   ↓
UpbitService
   ↓
Upbit REST API
   ↓
JSON
   ↓
UpbitTickerDto
   ↓
Controller
   ↓
브라우저에 JSON 출력
```

8. 다음 학습 단계

다음에는 현재가 1개가 아니라 XRP 최근 캔들 데이터를 가져와 MA5, MA20, RSI, 거래량을 계산합니다. 그 다음 계산 결과를 LangChain4j → Ollama → Gemma3로 전달하면 AI 시세 분석 구조가 완성됩니다.

```text
업비트 XRP 캔들
   ↓
최근 데이터
   ↓
MA5 / MA20 / RSI / 거래량
   ↓
LangChain4j
   ↓
Ollama
http://localhost:11434
   ↓
Gemma3
   ↓
상승 / 하락 / 횡보 분석
```

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.